Design and Analysis of Algorithm

Module V

Let me know, if any typos and other mistakes in the shared ppt slides

Module 5

Module No. 5	Backtracking	6 Hours
n-Queens problem (Four & Eight) – Hamiltonian Circuit Problem – Subset Sum Problem –Graph Coloring Problem		

Backtracking is a class of algorithms for finding solutions to some computational problems

Types of Backtracking Algorithm

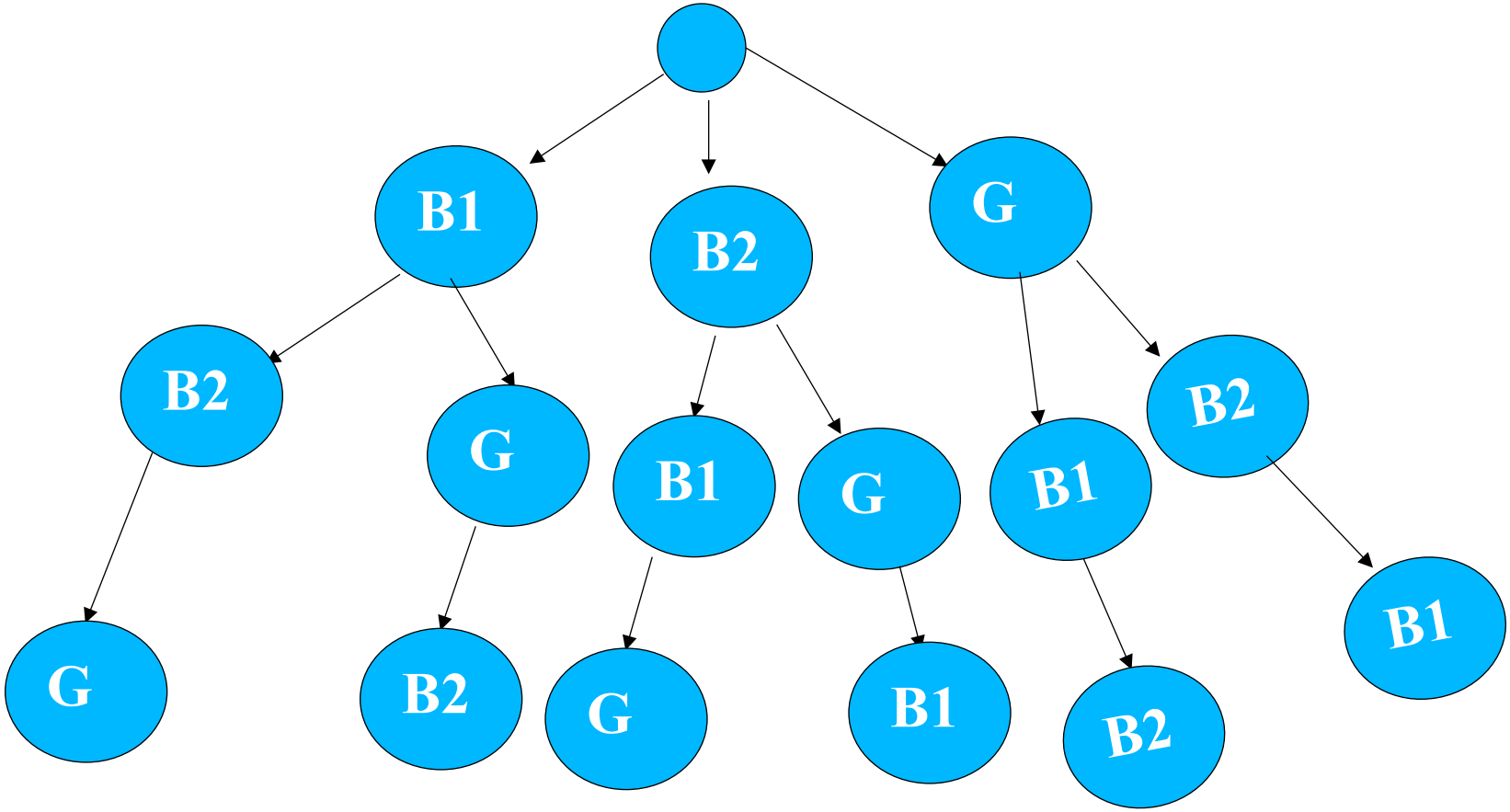
Backtracking algorithms are classified into two types:

1. **Algorithm for recursive backtracking**

2. **Non-recursive backtracking algorithm**

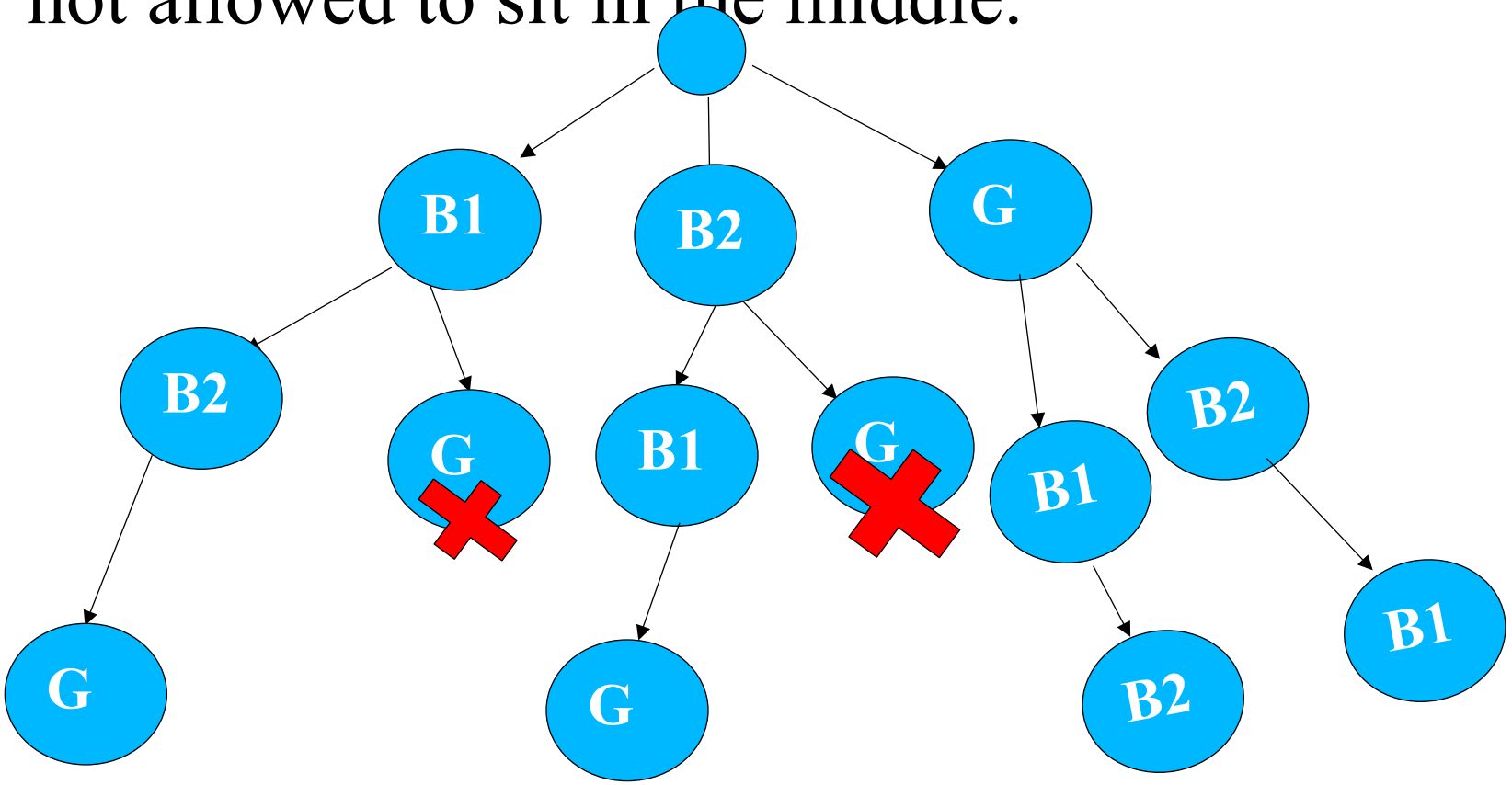
Example : State-Space-Tree

Sitting arrangement of 3 students(B1,B2,G) in a space table of size 3 without any constraints.



Example : State-Space-Tree

Sitting arrangement of 3 students(B1,B2,G) in a space table of size 3 with a constraint that girl is not allowed to sit in the middle.



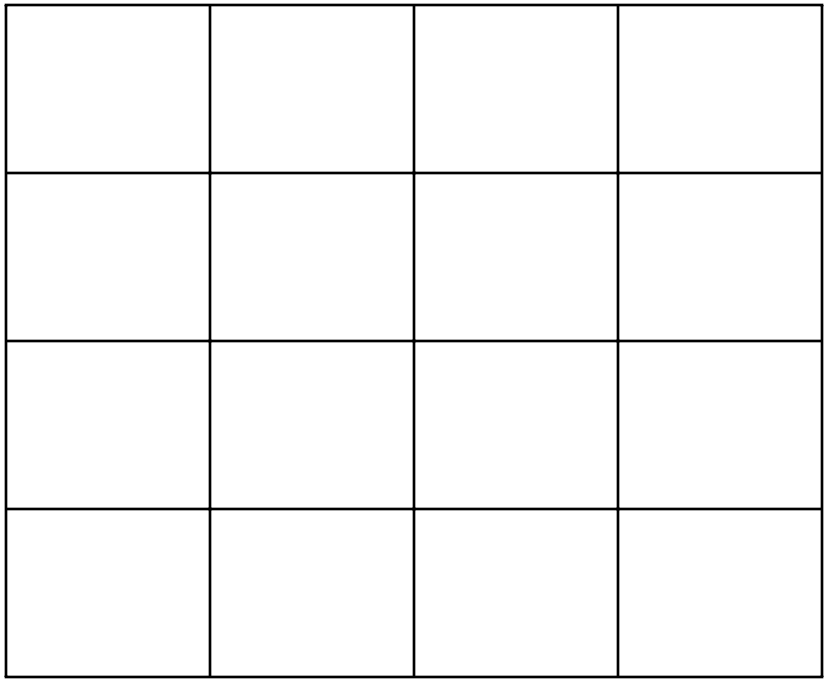
Backtracking Vs Branch &Bound

Backtracking → uses DFS

Branch & Bound → uses BFS

N-Queen Problem

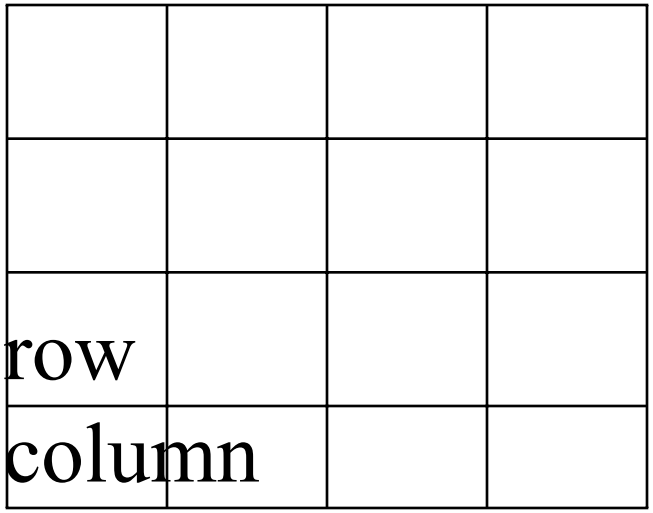
The **n-queens** puzzle is the problem of placing **n** queens on an **n** x **n** chessboard such that no two queens attack each other.



N-Queen Problem

Consider n*n chess board and try to find all ways to place n non-attacking queens

N=4



Constraints

No two or more queens in a row

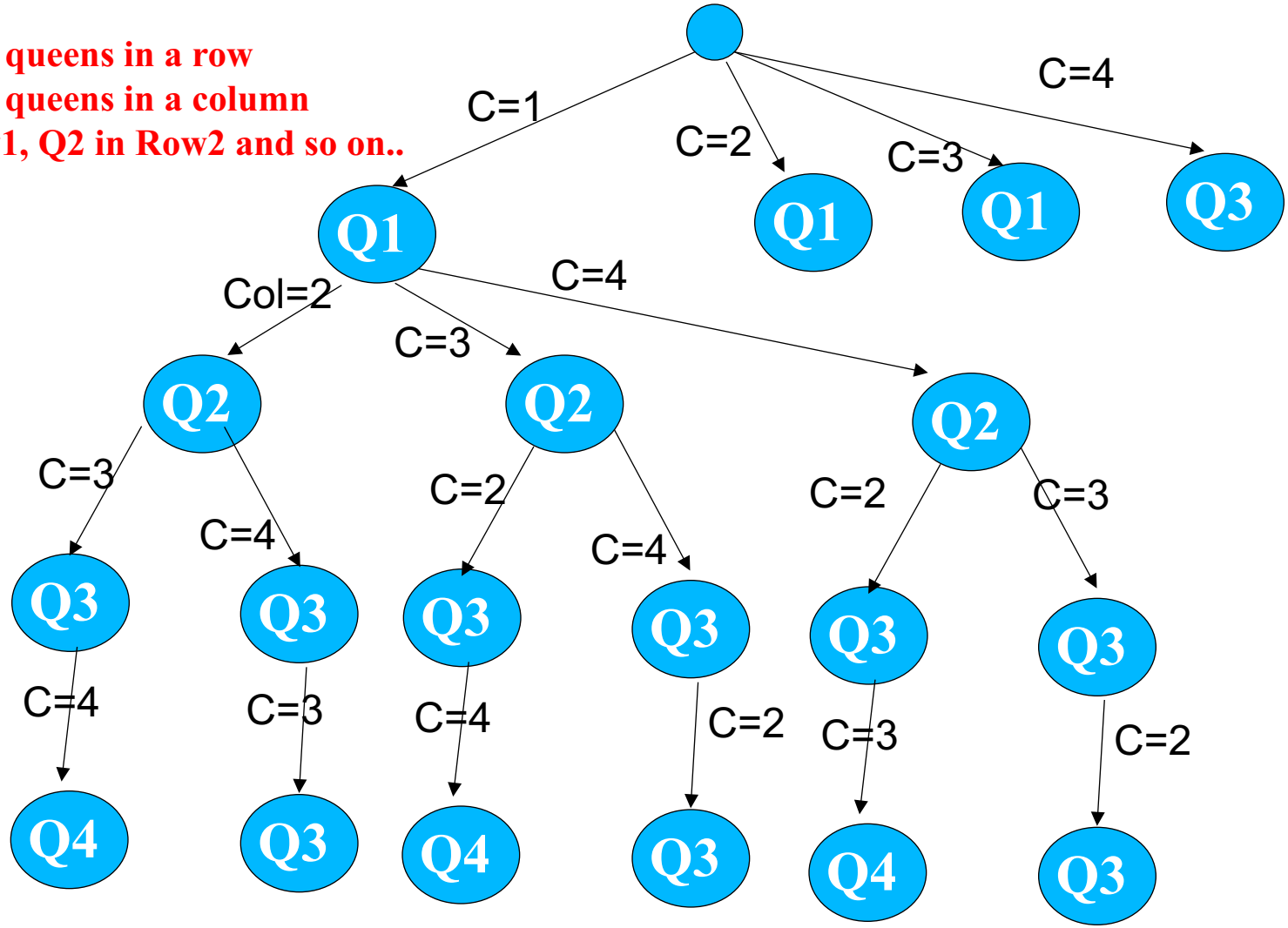
No two or more queens in a column

4-Queen Example

Constraints

- No two or more queens in a row
- No two or more queens in a column

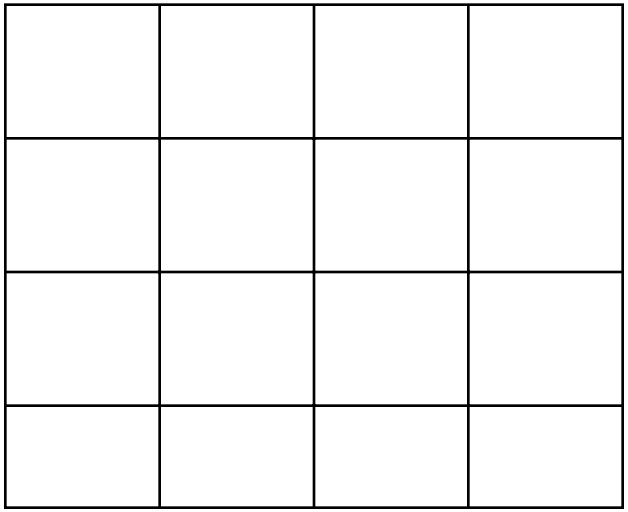
Note: Q1 in Row1, Q2 in Row2 and so on..

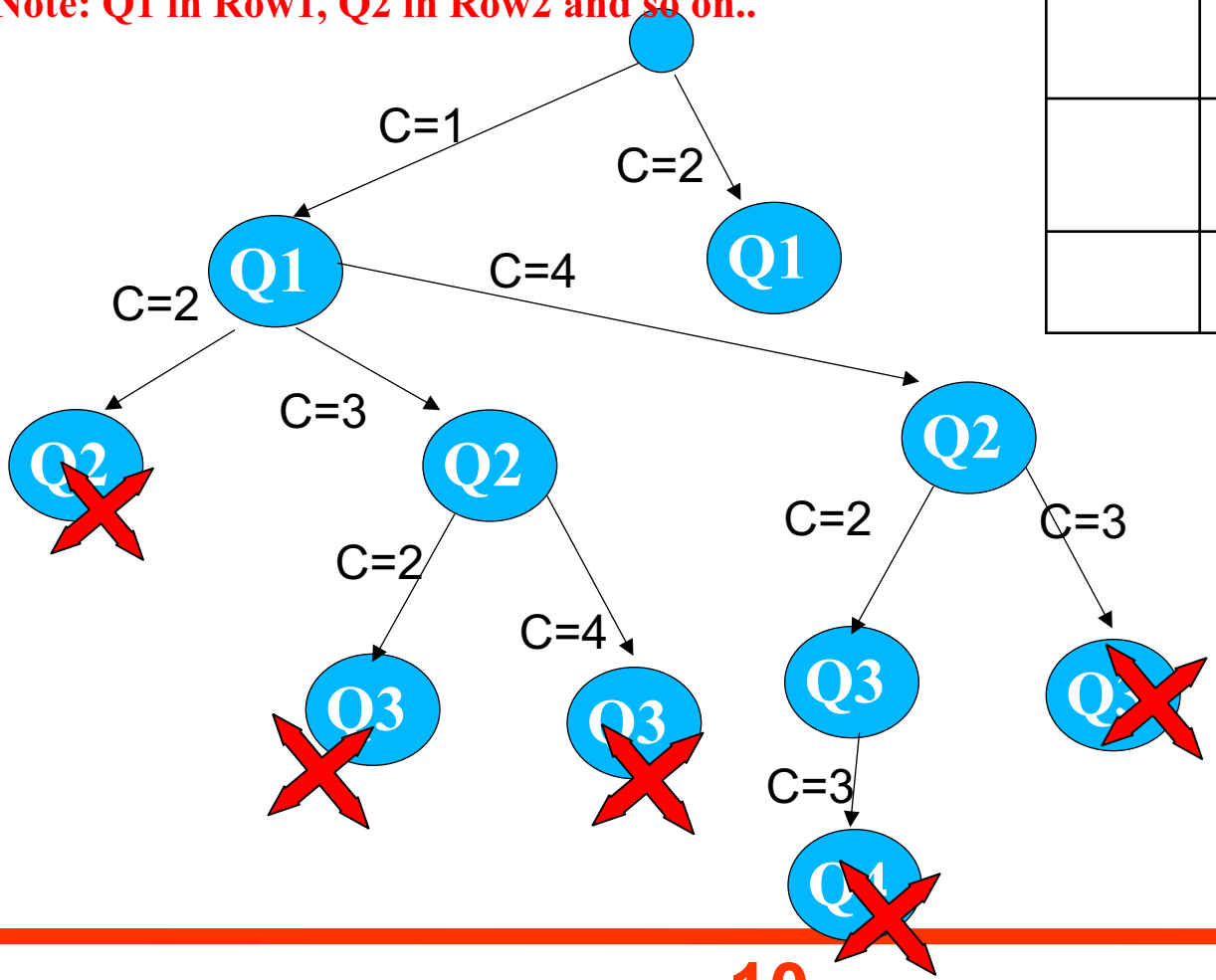


4-Queen Example

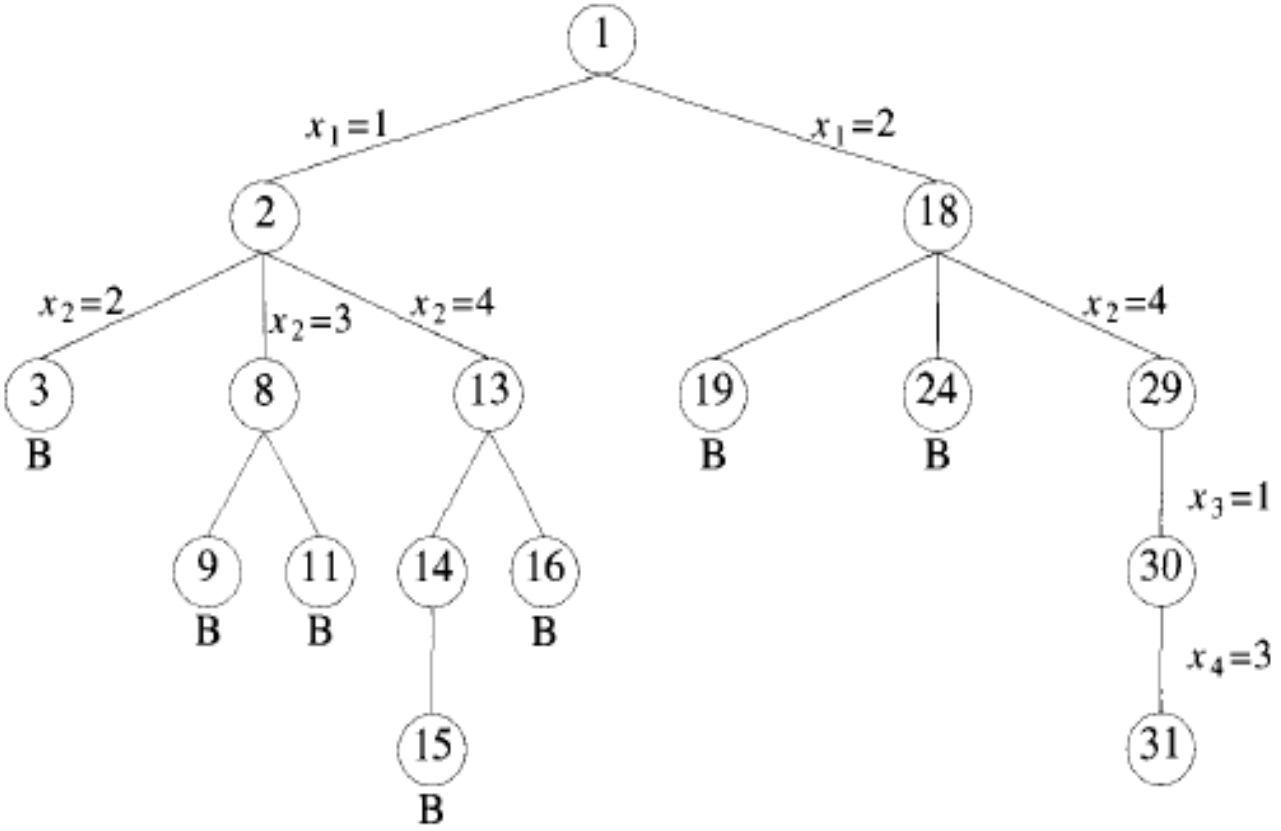
Constraints

- No two or more queens in a row
- No two or more queens in a column
- No two or more in same diagonal
- Note: Q1 in Row1, Q2 in Row2 and so on..





4-Queen Solution(State Space Tree)

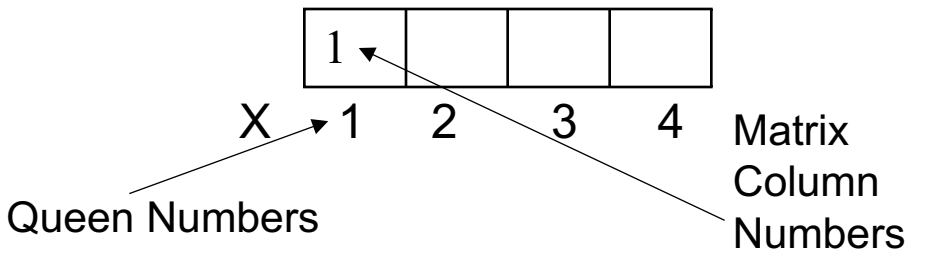


Algorithm

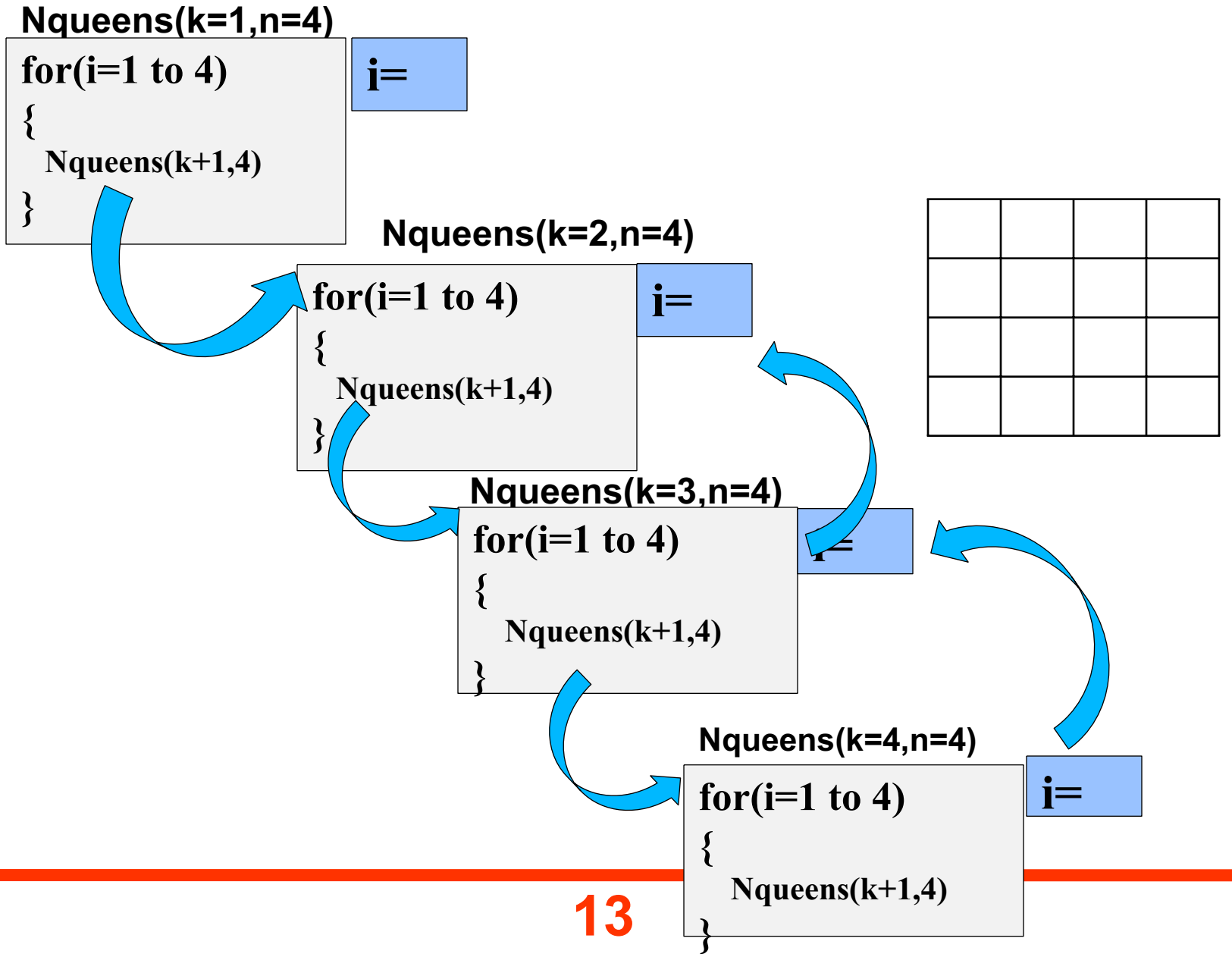

```
1   Algorithm NQueens(k, n)
2   // Using backtracking, this procedure prints all
3   // possible placements of n queens on an n × n
4   // chessboard so that they are nonattacking.
5   {
6       for i := 1 to n do
7       {
8           if Place(k, i) then
9           {
10              x[k] := i;
11              if (k = n) then
                    write (x[1 : n]);
12              else
                    NQueens(k + 1, n);
13          }
14      }
15  }
```

Row position are fixed here

```
Algorithm Place(k, i)
{
    for j := 1 to k − 1 do
        if ((x[j] = i)                          // Two in the same column
            or (Abs(x[j] − i) = Abs(j − k)))    // or in the same diagonal
            then return false;
    return true;
}
```



Recursive Execution



EXECUTION PROCESS

Nqueens(k=1,n=4)

```
for(i=1, 2, 3, 4)
{

}
```

X[1]= i (i.e. 1)

Nqueens(k=2,n=4)

```
for(i=1, 2, 3, 4)
{

}
```

False **False** **X[2]=3** **X[2]=4**

Nqueens(k=3,n=4)

```
for(i=1, 2, 3, 4)
{

}
```

Nqueens(k=3,n=4)

```
for(i=1, 2, 3, 4)
{

}
```

False

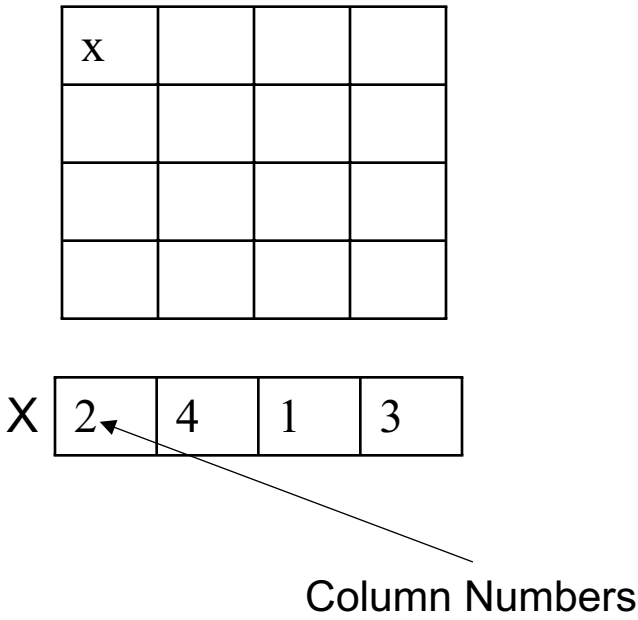
```
for i := 1 to n do
{
    if Place(k, i) then
    {
        x[k] := i;
        if (k = n) then
            write (x[1 : n]);
        else
            NQueens(k + 1, n);
    }
}
```

```
Algorithm Place(k, i)
{
    for j := 1 to k - 1 do
        if ((x[j] = i)        // Two in the same column
            or (Abs(x[j] - i) = Abs(j - k)))   // or in the same diagonal
            then return false;
    return true;
}
```

Q1			
		Q2	

NQueen(k,n)


```
void NQueen(int k,int n)
 {      int i;
        for(i=1;i<=n;i++)
        {       if(place(k,i)==1)
                {   x[k]=i;
                    if(k==n)
                       printboard(n);
                    else
                        NQueen(k+1,n);
                }
        }
 }
```



int place(k,i)


```
int place(int k,int i)
{ int j;
        for(j=1;j<k;j++)
        {       if((x[j]==i)||abs(x[j]-i)==abs(j-k))
                        return 0;
        }
return 1;
}
```

```
void printboard(int n)
 { int i;
        for(i=1;i<=n;i++)
                print(x[i]);
}
==================================
void main()
{ int n;
print("Enter Value of N:");
scan(n);
NQueen(1,n);
 }
```

Time complexity: O(N!) :
The first queen has N placements, the second queen must not be in the same column as the first as well as at an oblique angle, so the second queen has N-1 possibilities, and so on, with a time complexity of O(N!) .

Space Complexity: O(N) : Need to use arrays to save information.

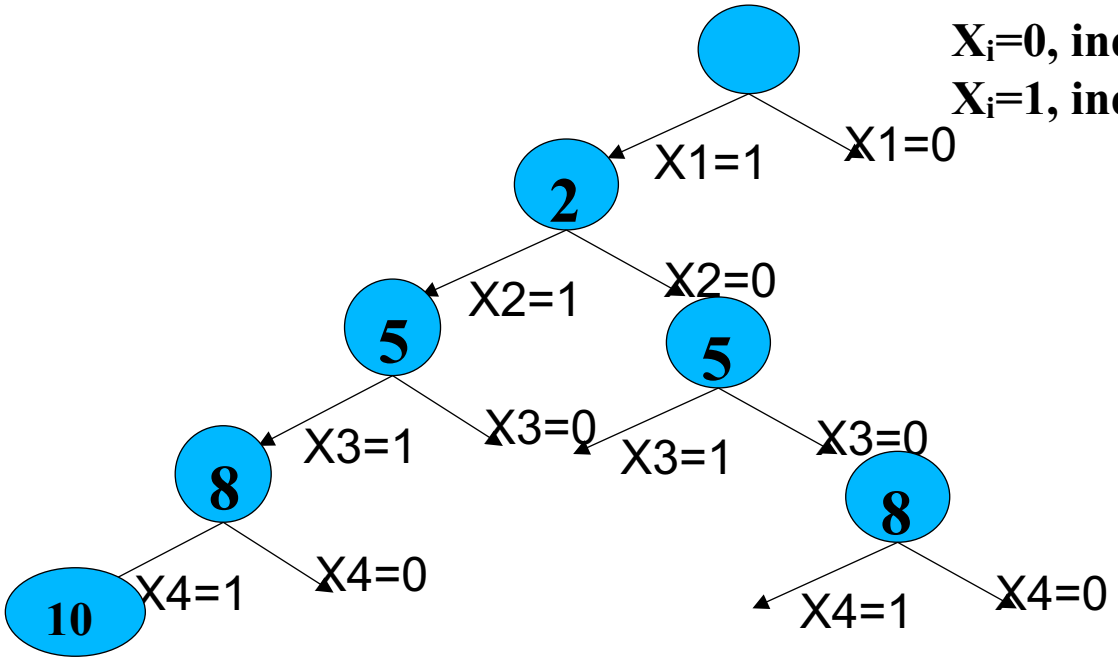
Subset Sum Problem

Subset Sum Problem

- Given n distinct positive integers, Desire to find all combinations of these numbers whose sum = m

- N=6
- M=30
- W[1 : 6]={5,10,12,13,15,18}

Example



X_i=0, indicates not included

X_i=1, indicates included in the solution

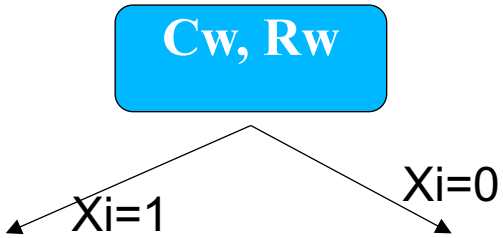
X	2	5	8	10

N=4 and M=15

If we continue for all possible solutions without a bounding condition, we will get 2^n solutions

Subset Sum Problem with Bounding

Cw: current weight
Rw: remaining weight



Xi=0, indicates not included
Xi=1, indicates included in the solution

X	2	5	8	10

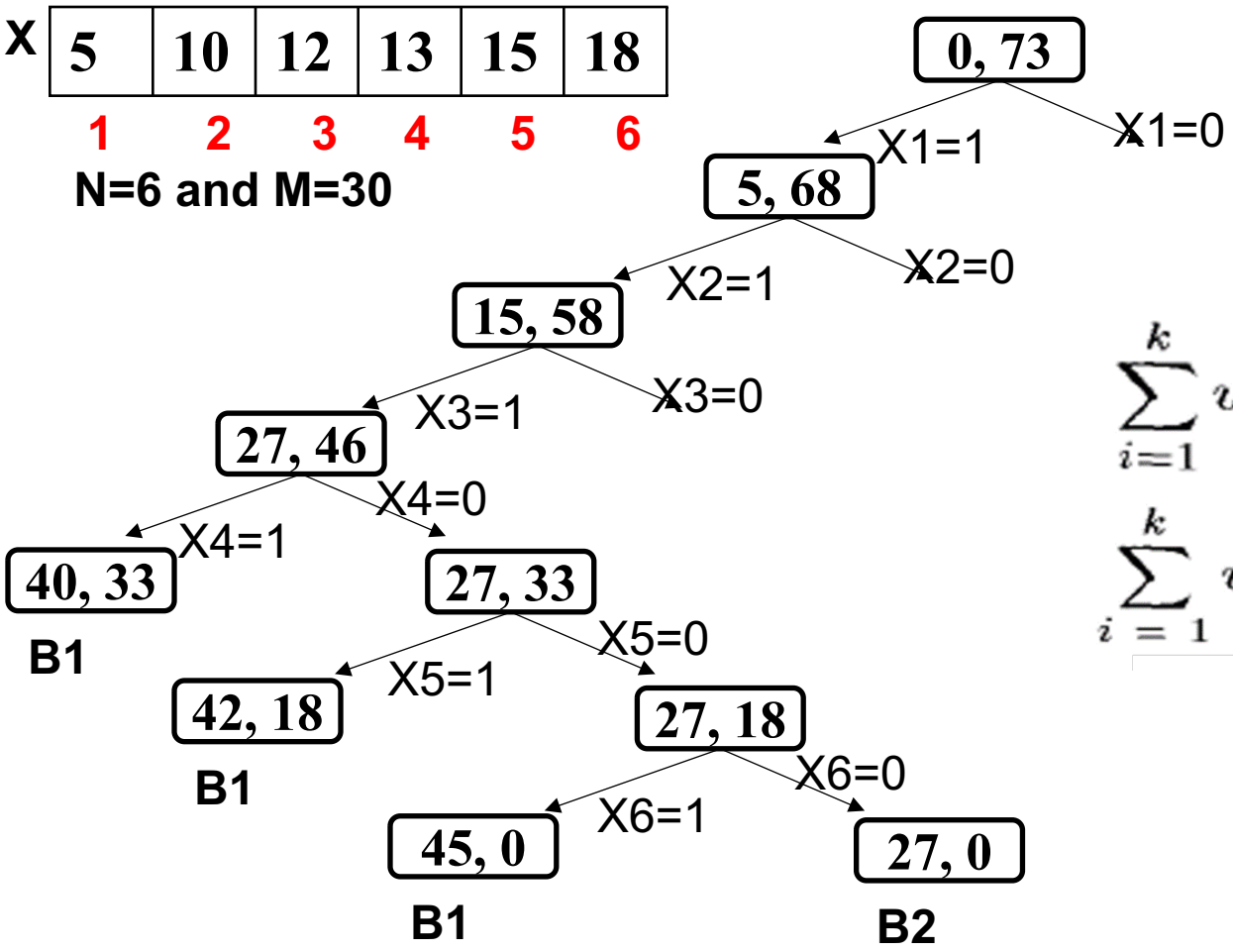
N=4 and M=15

Bounding Condition

$$\sum_{i=1}^{k} w_i x_i + w_{k+1} \leq m$$

$$\sum_{i=1}^{k} w_i x_i + \sum_{i=k+1}^{n} w_i > m$$

Example



Bounding Condition

$$\sum_{i=1}^{k} w_i x_i + w_{k+1} \leq m$$

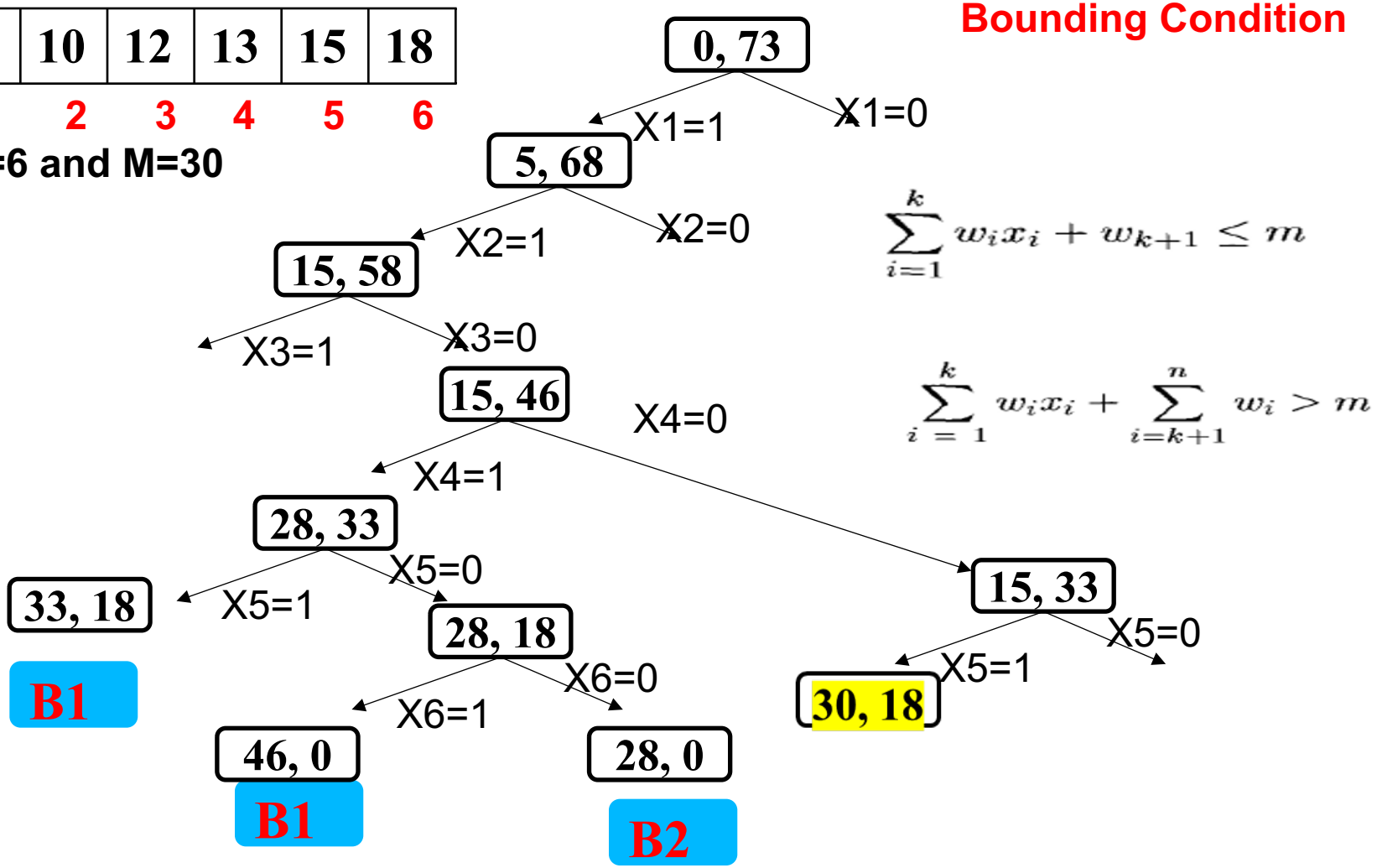
$$\sum_{i=1}^{k} w_i x_i + \sum_{i=k+1}^{n} w_i > m$$

Example

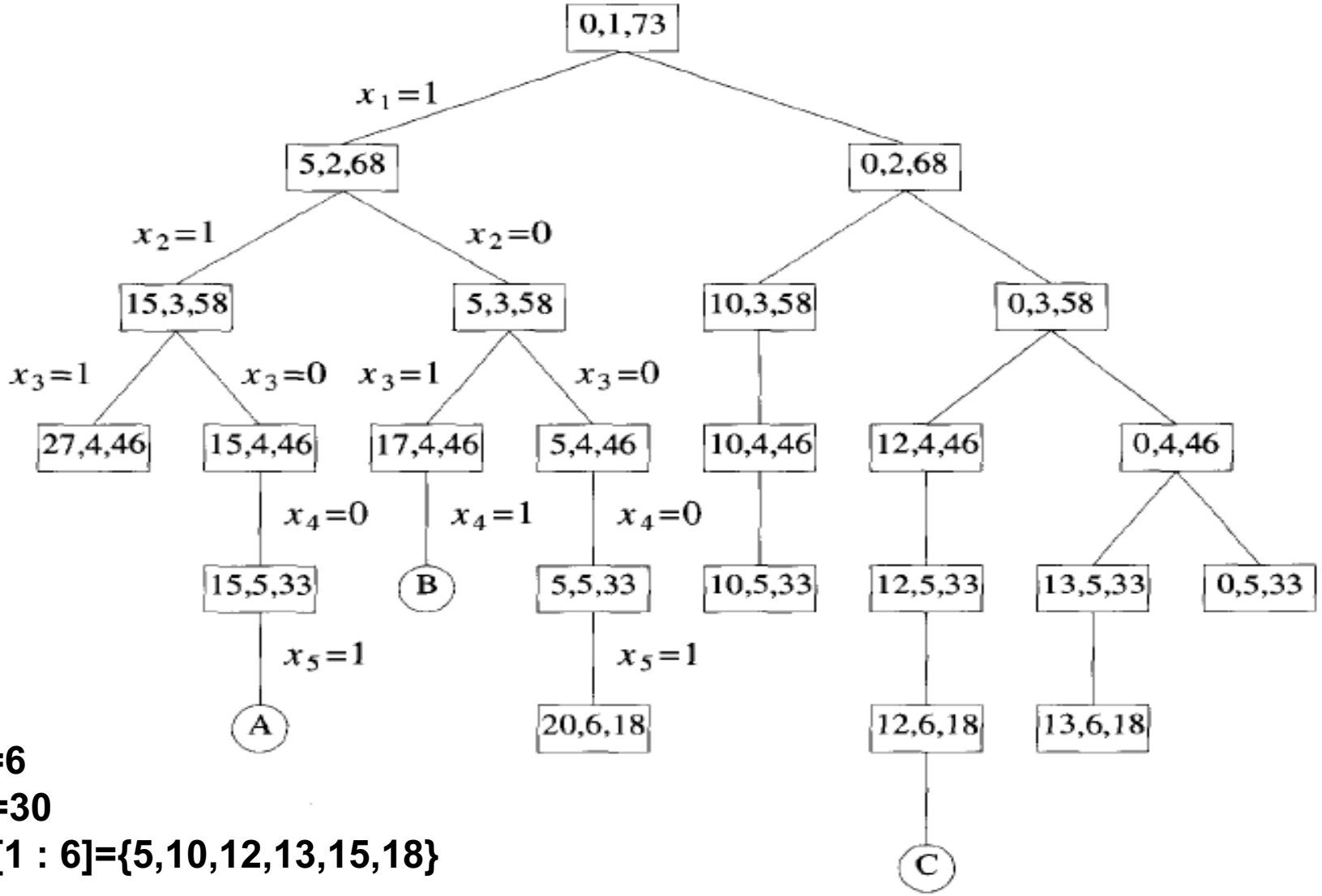
X	5	10	12	13	15	18
	1	2	3	4	5	6

N=6 and M=30

Bounding Condition



Example



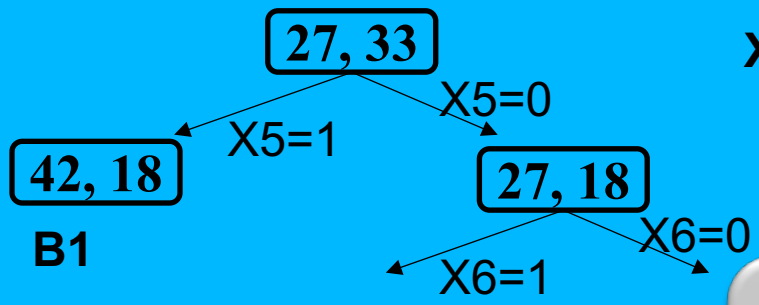
N=6

M=30

W[1 : 6]={5,10,12,13,15,18}

Algorithm


```
1   Algorithm SumOfSub(s, k, r)
2   // Find all subsets of w[1 : n] that sum to m. The values of x[j],
3   // 1 ≤ j < k, have already been determined. s = Σ_{j=1}^{k-1} w[j] * x[j]
4   // and r = Σ_{j=k}^{n} w[j]. The w[j]'s are in nondecreasing order.
5   // It is assumed that w[1] ≤ m and Σ_{i=1}^{n} w[i] ≥ m.
6   {
7       // Generate left child. Note: s + w[k] ≤ m
8       x[k] := 1;
9       if (s + w[k] = m) then write (x[1 : k]); // Subset found
10          // There is no recursive call here as w[j] > 0, 1 ≤ j ≤ n.
11      else if (s + w[k] + w[k + 1] ≤ m)
12          then SumOfSub(s + w[k], k + 1, r − w[k]);
13      // Generate right child and evaluate B_k.
14      if ((s + r − w[k] ≥ m) and (s + w[k + 1] ≤ m)) then
15      {
16
17
18
19  }
```



X	5	10	12	13	15	18
	1	2	3	4	5	6

N=6 and M=30

S+r-w[k]>=m
27+(33-15)>=30
Let assume
12+(30-15)>=30

```
1    Algorithm SumOfSub(s, k, r)
6    {
8        x[k] := 1;
9        if (s + w[k] = m) then write (x[1 : k]); // Subset found

11       else  if (s + w[k] + w[k + 1] ≤ m)
12                 then SumOfSub(s + w[k], k + 1, r − w[k]);

14       if ((s + r − w[k] ≥ m) and (s + w[k + 1] ≤ m)) then
15       {
16           x[k] := 0;
17           SumOfSub(s, k + 1, r − w[k]);
18       }
19   }
```

Recursive Execution Step=1

SumOfSub(s=0, k=1,r=30)

```
If(s+w[k]==m)
   print solution
Else if(s+w[k]+w[k+1]<=m)
      SumOfSub(s+w[k], k+1,r-w[k])
If(s+r-w[k]>m or s+w[k+1]<m)
   SumOfSub(s=0, k+1,r-w[k])
```

s=0+3

X	3	5	10	12
	1	2	3	4

N=4 and M=20

SumOfSub(s=3,k=2,r=27)

```
If(s+w[k]==m)
   print solution
Else if(s+w[k]+w[k+1]<=m)
      SumOfSub(s+w[k], k+1,r-w[k])
If(s+r-w[k]>m or s+w[k+1]<m)
   SumOfSub(s=5, k+1,r-w[k])
```

s=3+5

SumOfSub(s=8,k=3,r=22)

S=8+(10+12) <=20

NO

```
If(s+w[k]==m)
   print solution
Else if(s+w[k]+w[k+1]<=m)
      SumOfSub(s+w[k], k+1,r-w[k])
If(s+r-w[k]>=m or s+w[k+1]<=m)
   SumOfSub(s=8, k+1,r-w[k])
```

s=8+10

8+(22-10)>=20 &&
8+12<=20

Recursive Execution Step=2

SumOfSub(s=8, k=4,r=12)

```
If(s+w[k]==m)
   print solution
Else if(s+w[k]+w[k+1]<=m)
      SumOfSub(s+w[k], k+1,r-w[k])
If(s+r-w[k]>m or s+w[k+1]<m)
   SumOfSub(s=13, k+1,r-w[k])
```

s=8+12

X

5	8	10	12
1	2	3	4

N=4 and M=20

Recursive Execution Step=3

SumOfSub(s=13, k=4,r=12)

```
If(s+w[k]==m)
   print solution
Else if(s+w[k]+w[k+1]<=m)
      SumOfSub(s, k+1,r)
If(s+r-w[k]>m AND s+w[k+1]<m)
   SumOfSub(s=13, k+1,r-w[k])
```

s=13+12

13+(12-12)>20 or
13+12<20
Backtrack

X

5	8	10	12
1	2	3	4

N=4 and M=20

SumOfSub(s=5,k=2,r=30)

```
If(s+w[k]==m)
   print solution
Else if(s+w[k]+w[k+1]<=m)
      SumOfSub(s, k+1,r)
If(s+r-w[k]>m or s+w[k+1]<m)
   SumOfSub(s=5, k+1,r-w[k])
```

s=5+8

Backtrack

SumOfSub(s=13,k=3,r=22)

```
If(s+w[k]==m)
   print solution
Else if(s+w[k]+w[k+1]<=m)
      SumOfSub(s, k+1,r)
If(s+r-w[k]>m or s+w[k+1]<m)
   SumOfSub(s=13, k+1,r-w[k])
```

s=13+10

13+(22-10)>20 or
13+12<20

Complexity analysis:

Time Complexity: $O(2^n)$ worst case

The above solution may try all subsets of the given set in the. Therefore time complexity of the above solution is exponential.

Auxiliary Space: O(n) where n is recursion stack space.

```
public static void main(String args[])
    {
        int list[] = {1, 3, 5, 2};
        subset_sum(list, 0, 0, 6);
        System.out.println(subset_count);
    }
}
```

```
public class subset_sum_backtrack
{    static int subset_count = 0;

     static void subset_sum(int list[], int sum, int starting_index, int target_sum)
   {
      if( target_sum == sum )
      {
         subset_count++;
         if(starting_index < list.length)
            subset_sum(list, sum - list[starting_index-1], starting_index, target_sum);
      }
      else
      {
         for( int i = starting_index; i < list.length; i++ )
         {
            subset_sum(list, sum + list[i], i + 1, target_sum);
         }
      }
   }
```

Graph Coloring Problem

Graph Coloring Problem

The Graph Coloring Problem is a classic problem in graph theory. It involves assigning colors to the vertices of a graph such that **no two adjacent vertices** share the same color. The primary objective is to **minimize the number of colors used**.

Vertex coloring is the most commonly encountered graph coloring problem. The problem states that given m colors, determine a way of coloring the vertices of a graph such that no two adjacent vertices are assigned same color.

Objective

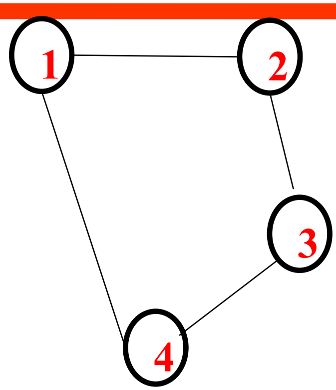
The objective is to find

→How many minimum number of colours are required to colour the graph nodes: **m-Colouring Optimization**

→ if **m** colours are given then, is it possible to colour the graph node with given number of colours or not:**-Graph colouring Algorithm/ m-Colouring Decision Problem**.

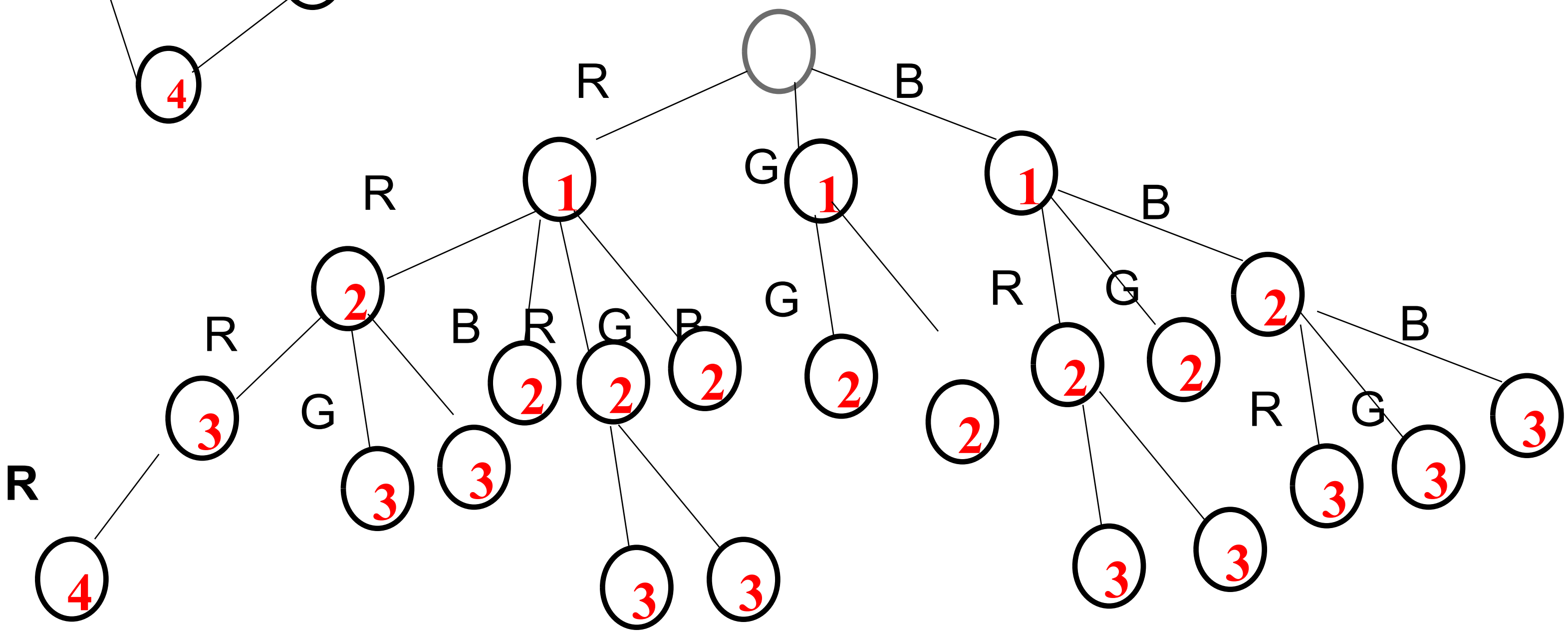
→

Solutions without Bounding Conditions



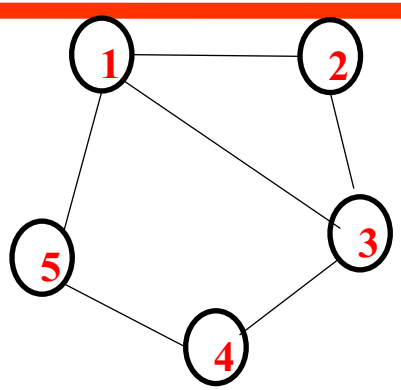
x	1	2	3	4	5

M=3 {Red, Green, Blue}



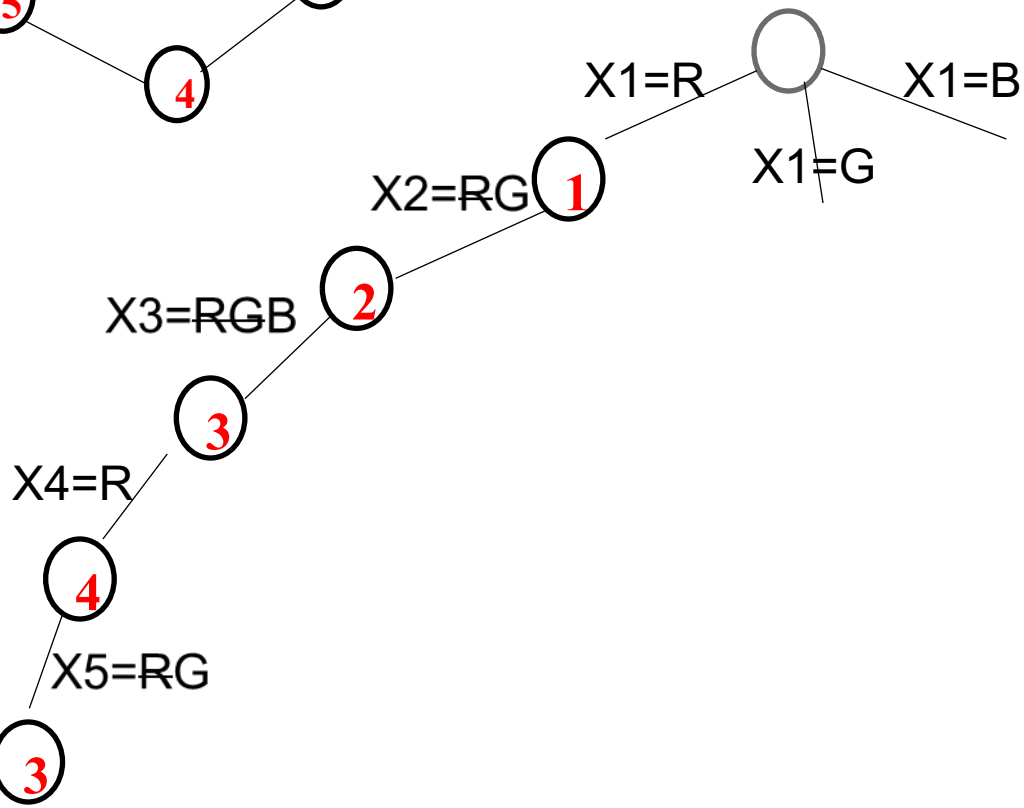
Number of solutions =$C^{(n+1)}$

Example

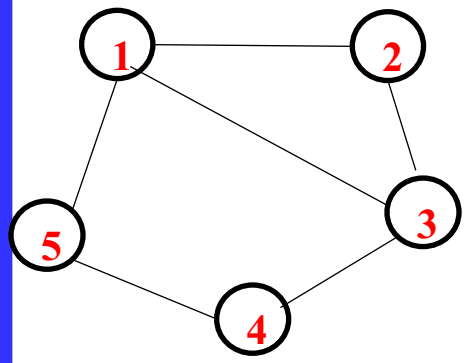


x	1	2	3	4	5

M=3 {Red, Green, Blue}



A Graph G with **n** nodes and **m** number of given colors. Initial value of k=1



0	1	1	0	1
1	0	1	0	0
1	1	0	1	0
0	0	1	0	1
1	0	0	1	0

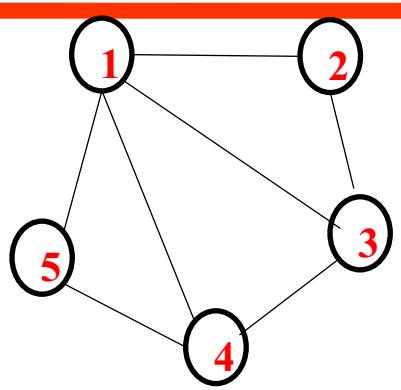
0,1	0,1,2	0,1,2,3	0,1	0,1,2
X, 1	2	3	4	5

M=3 {1=Red, 2=Green, 3=Blue}

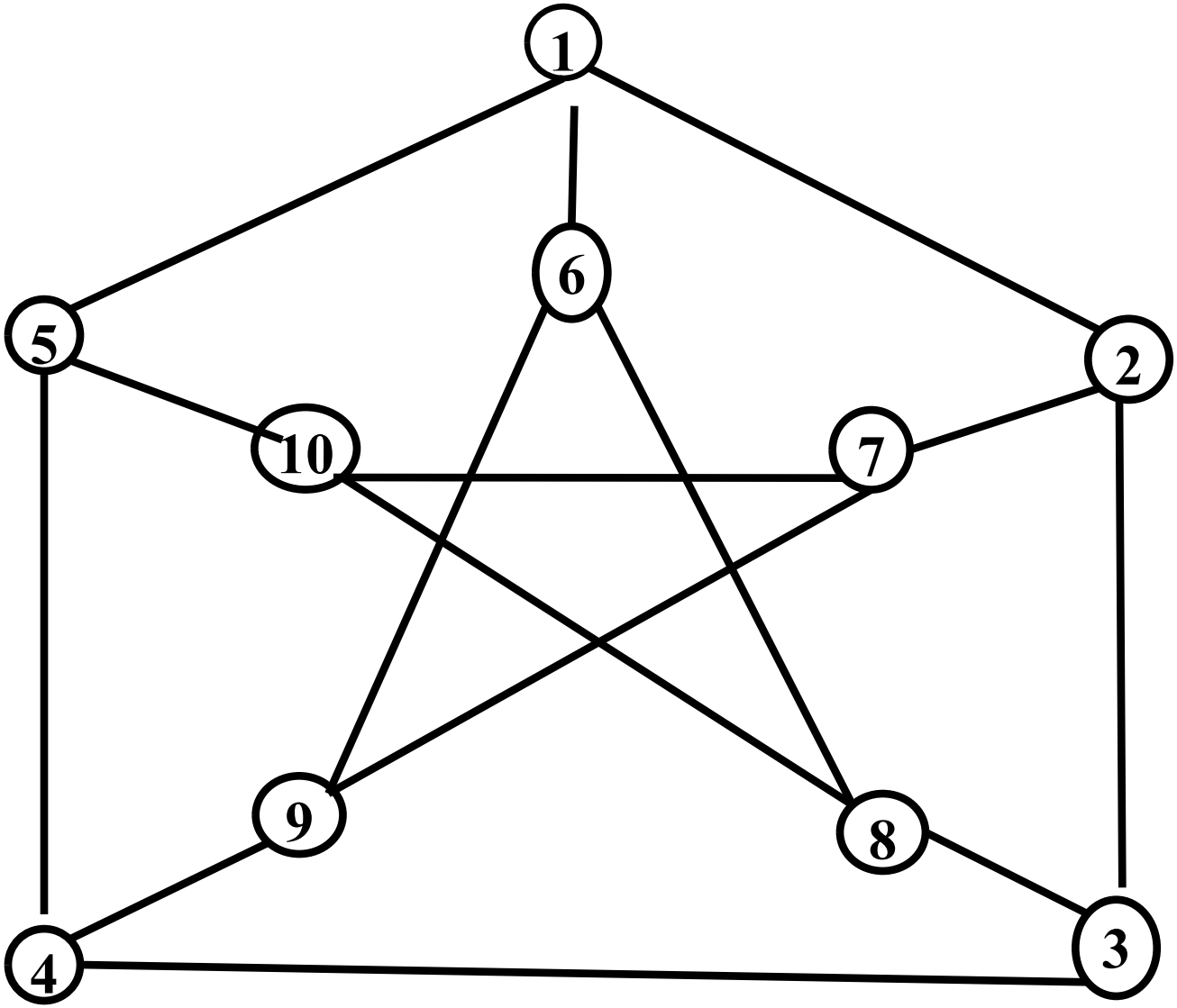
```
1.  Algorithm mcoloring (k)
2.  {
3.  repeat
4.  { // Generate all legal assignment for x [k]
5.  nextvalue (k);
6.  if (x[k]=0) then return; //no color possible
7.  if (k=n) then // at most m color have been used
8.  write (x[1:n]);
9.  else
10. mcoloring(k+1);
11. } until(false);
12. }
```

```
1.  Algorithm next value (k)
2.  {
3.  repeat
4.  {
5.  x[k]=(x[k]+1) mod (m+1); //next highest color
6.  if (x[k]=0) then return; //all colors have been used
7.  for j:=1 to n do
8.  {
9.  if ((G[k,j] ≠0) and (x[k]=x[j])
10. // adjacent vertices have the same color
11. then break;
12. }
13. if (j=n+1) then return; //new color found
14. }until (false); //otherwise try to find another color
15. }
```

Example



Example



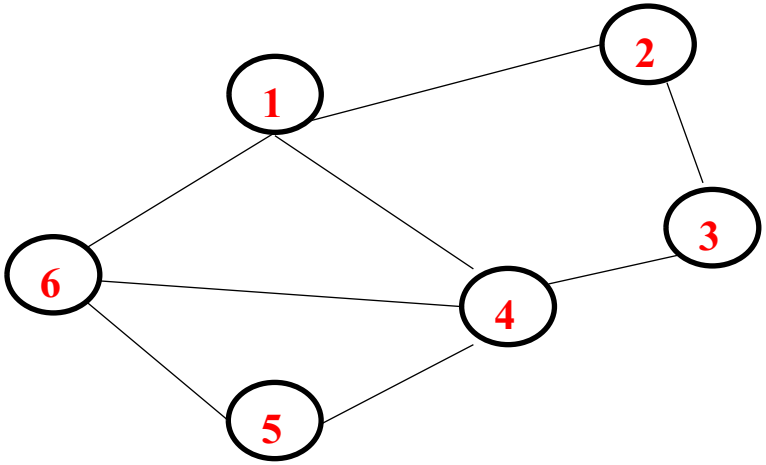
Hamiltonian Circuit Problem

A Hamiltonian circuit is a circuit that visits every vertex once with no repeats, and must start and end at the same vertex.

Removing any edge from a Hamiltonian cycle produces a Hamiltonian path

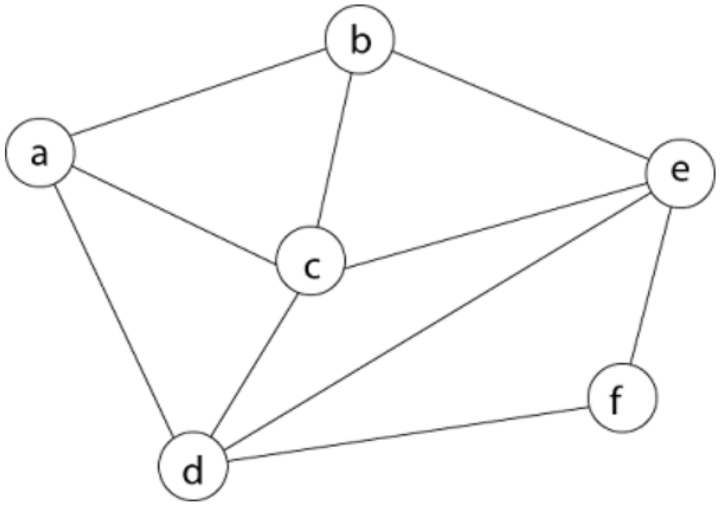
Hamiltonian Circuit Problem

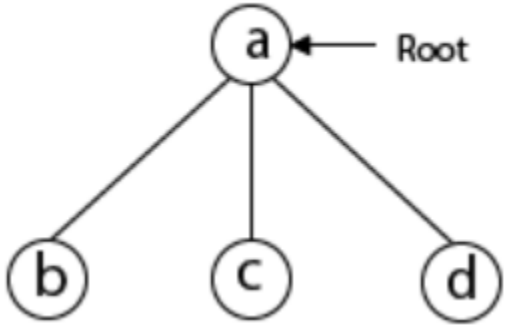
Let $G = (V, E)$ be a connected graph with n vertices. A Hamiltonian cycle (suggested by Sir William Hamilton) is a round-trip path along n edges of G that visits every vertex once and returns to its starting position. In other words if a Hamiltonian cycle begins at some vertex $v_1 \in G$ and the vertices of G are visited in the order $v_1, v_2, \ldots, v_{n+1}$, then the edges (v_i, v_{i+1}) are in E, $1 \leq i \leq n$, and the v_i are distinct except for v_1 and v_{n+1}, which are equal.

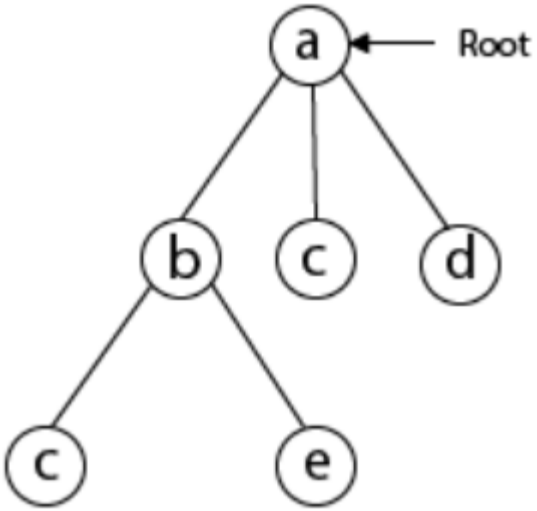


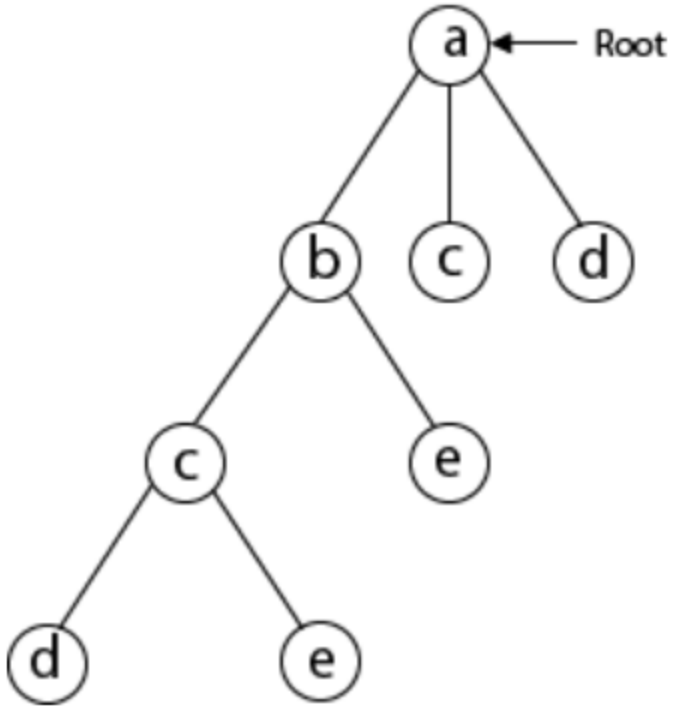
0	1	0	1	0	1
1	0	1	0	0	0
0	1	0	1	0	0
1	0	1	0	1	1
0	0	0	1	0	1
1	0	0	0	1	0

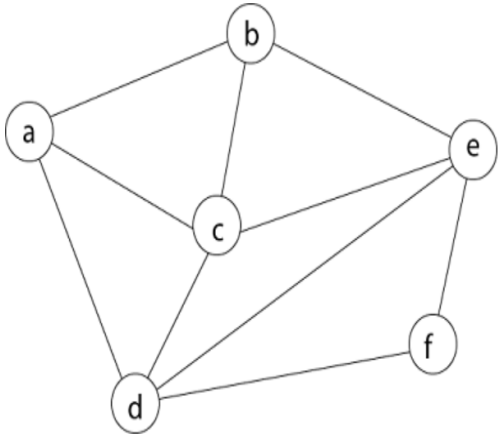
Example



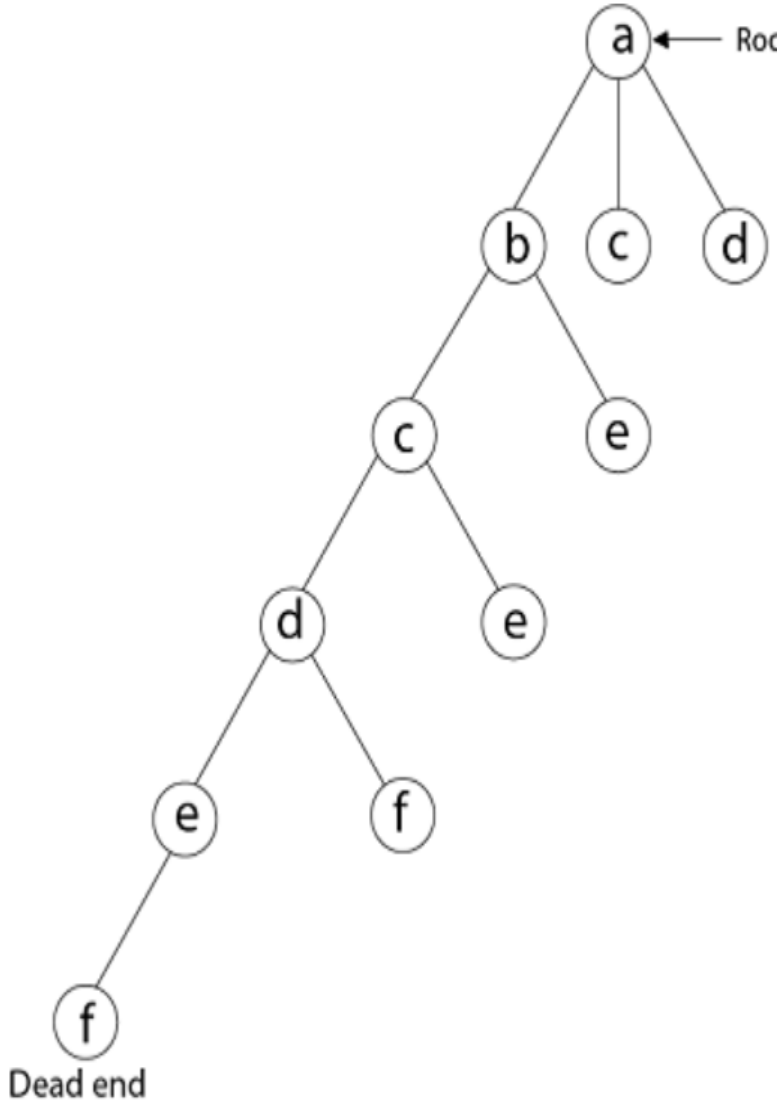


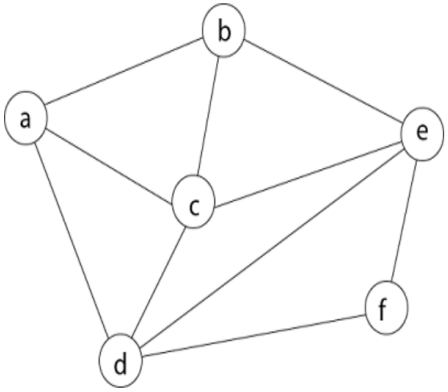





a
b
c
d
e
f

a
Root
b
c
d
c
e
d
e
e
f

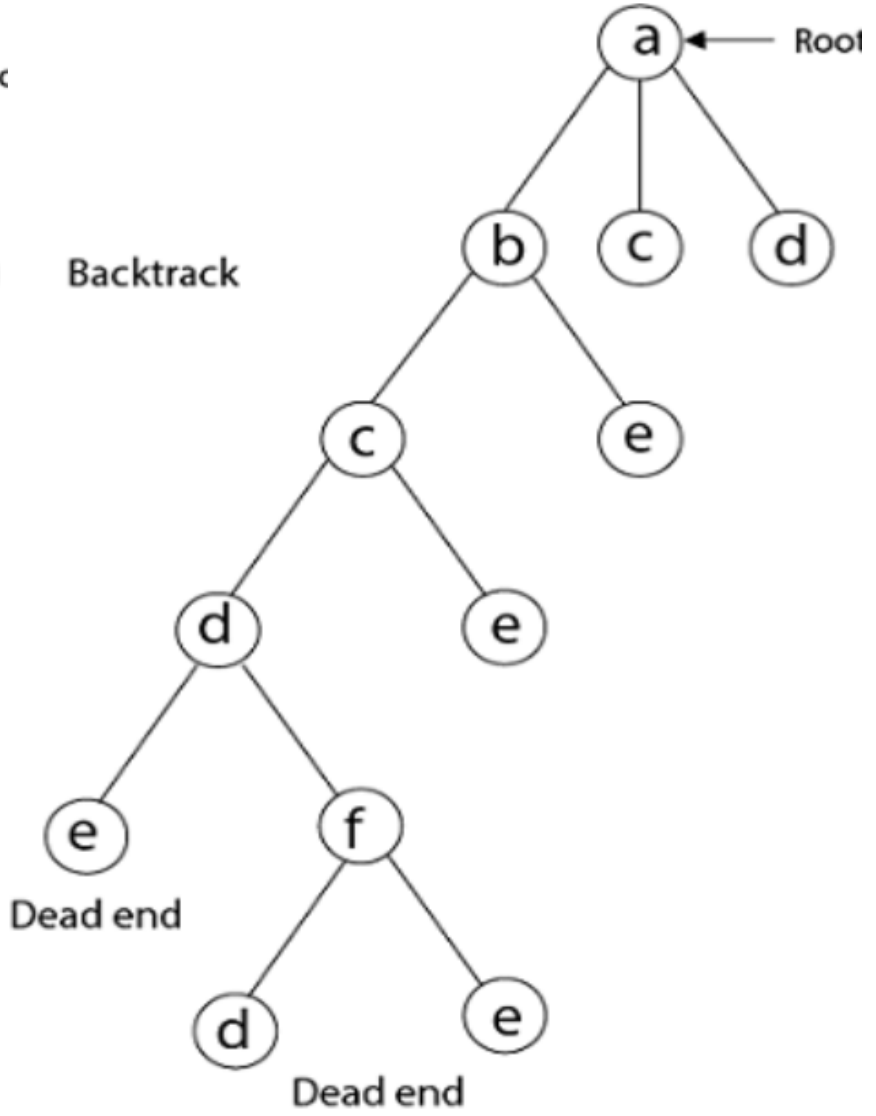

a
Roc
b
c
d
c
e
d
e
e
f
f
Dead end



Backtrack

Dead end

Backtrack



Dead end

Dead end

Algorithm


```
1   Algorithm Hamiltonian(k)
2   // This algorithm uses the recursive formulation of
3   // backtracking to find all the Hamiltonian cycles
4   // of a graph. The graph is stored as an adjacency
5   // matrix G[1 : n, 1 : n]. All cycles begin at node 1.
6   {
7       repeat
8       { // Generate values for x[k].
9           NextValue(k); // Assign a legal next value to x[k].
10          if (x[k] = 0) then return;
11          if (k = n) then write (x[1 : n]);
12          else Hamiltonian(k + 1);
13      } until (false);
14  }
```

k starts with 1

Initially x[i]=0

0	0	0	0	0	0	0

Algorithm


```
Algorithm NextValue(k)
// x[1 : k − 1] is a path of k − 1 distinct vertices. If x[k] = 0, then
// no vertex has as yet been assigned to x[k]. After execution,
// x[k] is assigned to the next highest numbered vertex which
// does not already appear in x[1 : k − 1] and is connected by
// an edge to x[k − 1]. Otherwise x[k] = 0. If k = n, then
// in addition x[k] is connected to x[1].
{
     repeat
     {
          x[k] := (x[k] + 1) mod (n + 1); // Next vertex.
          if (x[k] = 0) then return;
          if (G[x[k − 1], x[k]] ≠ 0) AND if(k>1)) then
          { // Is there an edge?
               for j := 1 to k − 1 do if (x[j] = x[k]) then break;
                                    // Check for distinctness.
               if (j = k) then // If true, then the vertex is distinct.
                    if ((k < n) or ((k = n) and G[x[n], x[1]] ≠ 0))
                         then return;
          }
     } until (false);
}
```

1	0	0	0	0	0	0

K x[i]=0

Algorithm

```
Hamilton(x[ ],k)
{   do{
        nextValue(x[ ], k);
        if(x[k==0)
            return;
        if(k==n)
             write(x[],n)
        else
            Hamilton(x[], k+1)
    while(True);
}
```

```
nextValue(x[ ],k)
{   do {
        x[k]=x[k]+1 mod (n+1);
        if(x[k]==0)
                return;
         if(G[x[k-1], x[k] ] ==1)
            for(j=1; j<k; j++)
                if(x[k]==x[j])
                        break;
        if(j==k)
          if((k<n) OR (k==n &G[ x[k], x[1] ]==1))
                return;
      }while(True);
}
```

The time complexity of Hamiltonian Circuit is O(N!), where N is the number of vertices.

Backtracking: Has a worst-case time complexity of (O(N!) and a space complexity of (O(N).

Dynamic Programming: Has a time complexity of O(N^{2} * 2^{N}), which is better than O(N!) for larger values of (N), but it is memory-intensive.